



The Lazy Developer

09

MODULE

Setting up multiple environments (Dev > Staging > Prod)

This is where some real magic takes place. Make edits in dev, push them to staging for verification, and then make that final push to deploy and let the automatic server rebuild happen with ease.

Table of Contents

- 01. More complex apps require more vetting and care. (8 min)
- 02. Testing with Preview Deployments in a Staging Environment (8 min)
- 03. Additional considerations for a complex environment. Tips and gotchas! (10 min)

More Complex Apps Require More Planning and Looking Ahead

The real secret to building production-ready apps isn't just writing great code but it's setting yourself up with a strong development process. Before you even start building, it helps to think through where your code will live, how you'll test it, and how you'll know it's safe to release.

For a small business or a solo builder using AI tools like Cursor/Claude Code, this can feel like overkill but it's not. It's how you protect yourself from late-night emergencies, broken features, and costly bugs in production.

Why This Matters (Especially for Small Teams)

Let's say you're building a membership site for a fitness coach. You start out just testing it on your laptop: logins work, videos play, payments seem fine. So you go ahead and push it live.

A few weeks later:

- Your real users start getting auth errors. (Due to a single SQL query you ran)
- Payments are being duplicated, or are they? You can't tell. (Thanks to an update in your webhook from Stripe)
- The workout video player stops working on mobile. (You did a security assessment and made your content security policy far too strict)
- You realize some things only broke after you made "quick fixes" late one night. (And wake up to emails and a host of pain to start your day!)

Now, you're scrambling to fix it while customers are emailing in. That's stress you didn't need and it all comes down to having no separation between development, staging, and production. You should be focusing on building! Not getting sidetracked wasting precious hours trying to squash more bugs you didn't anticipate you would get.

If you had planned out those environments ahead of time, you'd have caught most of those issues in a safer space first.

Real-World Example: A Planned-Out Architecture

Let's walk through one of our real apps - Quiki - and how it was structured on day one to handle growth and change.

Development Environment (Dev)

- Runs locally using Docker Desktop
- Uses a local Postgres database, not a shared cloud one

- Still connects to Supabase for auth, so we're testing the real login flows
- The frontend runs with Vite for fast reloads

This lets us:

- Work without needing an internet connection
- Avoid paying for extra cloud resources during early dev
- Test real auth flows without touching production data
- Do large refactors without breaking anything live

ðŸŒ™, Staging Environment

- Supabase (auth + Postgres): hosted and persistent
- Frontend deployed on Vercel: so we can share preview links

Now we can:

- Push changes to a "staging" branch in GitHub
- Automatically get a real URL to test the app in production-like conditions
- See how it performs online, with real services

ðŸŒ™ðŸŒ™ Production

- Identical to staging, just connected to real users and payments

This setup gives us confidence. If it works in staging, it'll work in prod. We're not guessing. The amount of times we get to staging and we realise that didn't go as expected due to the distributed architecture was a real eye-opener. Having the staging area has made that effortless. As has the ability to simply roll back to a previous build instantly in Vercel or Railway.

What Each Environment Is Really For

Let's expand on what these environments actually do for you.

ðŸŒ™' Development (Dev)

- Your personal lab
- You can break things, restart services, change database structures, and no one sees it
- Perfect for large updates, experiments, or integrating new tools. And of course, those big new features.

Example: Trying out a new onboarding flow or switching to a new design framework? Do it here first.

ðŸŒ™>ðŸŒ™ Staging

- Your pre-launch testing ground
- Meant to mirror production as closely as possible (same auth, same database structure, etc.)
- Used to validate changes before releasing them live. Keep logging active here so you can validate all is working well.

Example: Before launching a new checkout flow, you'd push it to staging and try every edge case: expired card, discount code, mobile view, etc.

🚀 Production

- The real deal
- Users see it, data is live, emails and payments are triggered
- Only ship changes here once you're confident they've been tested in staging

How Planning Ahead Saves You (A Lot)

Without this setup, you'll find yourself:

- Copy/pasting changes into prod and hoping nothing breaks
- Making changes in staging and forgetting to sync them to dev
- Getting surprised by bugs that only show up online
- Having to halt development while you fix broken prod builds

When you've planned ahead:

- You avoid "Oh no!" moments after hitting deploy
- You save money by only paying for hosting once your app is stable
- You can move faster, because you're not constantly worried about breaking things
- You can build trust with clients, users, and even yourself

Note - If you're building a SPA (i.e. a website or portfolio site), it's safe to say you don't need a staging area due to the simplicity of the site. Simply having a local environment and your repo in Github connected to Vercel is enough. The changes you make are mostly cosmetic rather than than features or build out. Also, it's quite likely you have no authentication (aka "auth") so it simplifies the architecture considerably.

Prompt Ideas to Help Plan This

Here are some things you can ask AI to help you figure this out early:

- "Help me plan the environment setup for a SaaS app with auth, Postgres, and a React frontend."
- "Write a README section that explains how to run the app locally, in staging, and in production."

- “Create a GitHub branching strategy for dev, staging, and production workflows.”
 - “Configure a .env file structure to support local dev, staging, and prod environments separately.”
-

Final Thought

You don't need an enterprise team to work like a professional. Just putting a little thought into your environments upfront will make your entire app journey smoother from first prototype to paying customers. And the best part is AI can help guide you through all of it, step-by-step. As always, just ask!

This section is where your team (or just you!) starts to shift into a more professional workflow: pushing changes to a staging branch, seeing them appear in a real hosted environment, and testing everything as if it were live without the risk.

Testing with Preview Deployments in a Staging Environment

Once you've planned your environments and set up your GitHub branches (like dev, staging, and main), the next big step is preview deployments. This is how you test your app in a real, hosted environment before sending it to production, just to make sure all is working just as expected before pushing to production.

Tools like Vercel and Railway make this incredibly easy: every time you push changes to a certain branch (like staging), they automatically build your app and give you a live link to test it.

What Is a Preview Deployment?

A preview deployment is a full, online version of your app tied to a specific branch in your GitHub repo that you can visit, click around, and test like a real user.

For example:

- You push code to your staging branch
- Vercel detects the change
- It builds your frontend and gives you a live URL like:
`https://your-app-staging.vercel.app`
- You click the link, and bam — your new feature is running live, but only in staging

This is game-changing because it lets you:

- Share your work with teammates or clients before releasing
 - Catch layout issues, mobile bugs, or loading problems early
 - Safely test auth flows, form submissions, and logic
 - Debug in a real environment (not just locally)
-

How This All Fits Together

Let's walk through the full workflow:

1. You finish a feature in your dev branch.
2. You merge or push it to staging (in GitHub or via AI prompts).

3. Your deployment platform (like Vercel or Railway) detects the update, builds the app, and gives you a live link.
 4. You test the preview deployment:
 5. Everything looks good? Then you're ready to create a pull request in Github from staging into main and go live. That will then redeploy your instances that are "prod" that are connected to your "main" branch (as shown in the image above).
-

Real-World Example

Let's say you're adding a new pricing page to your app with 3 new plans. You're using Supabase for auth and Vercel for frontend hosting.

In staging, you want to:

- Test how the pricing page looks on mobile
- Try signing up with each plan and confirming the data goes to Supabase correctly
- Make sure the checkout buttons aren't accidentally pointing to your live payment system

With preview deployments, you can test all of this safely, without risking real customer data or triggering real charges.

How to Set This Up (AI-Friendly Version)

Here's what you need in place:

1. Connect Your GitHub Repo to a Platform Like Vercel or Railway

- In Vercel, you can select which branch to auto-deploy (choose staging)
- Every push to that branch will build and host your app on a new URL

Above is the Vercel "Environments" section in the "Settings" of your project. Right now, I have any branch getting picked up for a preview build, but if you have created a 'staging' branch, you could easily change this to point to staging for the safest approach. Just make sure you create the branch with AI or in Github first. If you've forgotten how to do that, ask AI or look at the resources on your Dashboard here on this site.

2. Testing Tips Once It's Live

When the preview link is ready, go through a simple checklist:

- ' Does it load on mobile and desktop?
- ' Can I sign up and log in with a test user?
- ' Do buttons and forms behave the way I expect?

- ' Are API calls going to the staging database, not production?
- ' Is anything throwing errors in the browser console?

You can even use AI to help:

"Help me write a staging test checklist for a feature release with auth, database, and email confirmation."

Why This Is a Game-Changer

Before preview deployments, you had to:

- Deploy changes manually to a staging server
- Spend hours syncing databases and resetting test users
- Hope your tests matched production

Now? You push code to a branch, and within a minute, you get a fresh version of your app to test online with zero setup.

Reccap

- Create your repo in Github at the VERY beginning
- Create your code branch, staging branch and other branches you may want to use.
- Head over to Vercel/Railway, sign up, connect to your repo. You can also use your GitHub login if needed.
- Setup a new project
- Connect preview to staging, and main to produciton.

That, in its simplest form, is the process. Get stuck along the way? AI can help you if you provide a screenshot or two to get you on the right track.

At first, deploying a frontend-only app with Vercel and some managed services feels almost magical. But as your app gets more complex with features like handling logins, interacting with a database, processing payments, then you need to start thinking about infrastructure, architecture, and trade-offs. Costs, performance, and flexibility all come into play.

This is where a bit of foresight can save a ton of pain later. As you read this section, keep in the back of your mind, this website runs entirely on Railway. The frontend client and server are two separate instances on Railway that talk to each other. Supabase handles the authentication and also all the course and user content, the purchase history, and all account information. There is extensive Row Level Security, Security Headers, security hardening, etc. Text doesn't take up a lot of storage, and neither do the images. The audio files only slightly do but are compressed before uploaded. Hopefully that helps you understand a bit more about what's involved here. 3 instances in total.

Additional Considerations for a Complex Environment

So far, we've talked about Dev !' Staging !' Production setups, preview deployments, and testing workflows. But how do you know when it's time to take things further? When does your app need more than just Vercel and Supabase? What happens when you hit limitations or need more control?

Let's explore when and why you'd start layering in more infrastructure and the real-world tradeoffs to consider.

When Do You Need a More Complex Setup?

Not every app needs to spin up its own servers or run custom backends. But the moment your app starts moving beyond static pages and into user logic, data processing, or tight performance requirements mean it's time to evaluate.

Here are common signals that you might need more:

' You're building a single-page application (SPA)

Modern SPAs often use frameworks like React or Svelte, and they rely heavily on APIs and dynamic routing. You'll need to think about:

- Caching and revalidation
- Real-time updates
- Browser vs. server rendering
- How your auth logic works in a client-rendered app

Vercel is great for SPAs, but once you start needing custom backend logic, that may need to move elsewhere.

' You're storing and manipulating user data

Even if you're using Supabase, once you're dealing with user data (profiles, uploads, custom dashboards), you're now responsible for data consistency, security, and performance. Not to mention adhering to GDPR rules!

You'll start asking:

- How do I seed different databases for staging vs. prod?
- What sort of security do I need? (Go back to the Security Module!)
- Where do I go to access the logs across all these environments?

' You have authentication and permissions

Anything with user logins, roles (admin, editor, viewer), or gated content raises the complexity quickly.

- Supabase helps here, but integrating auth in a real-world app still means wiring it into the UI, backend, and database correctly. Thankfully Cursor can help with all that! However, you will need to consider what is behind a user login and what isn't. How to protect the authenticated content. How to stop one user from accidentally accessing another users' information, etc.
- You might want to mock certain users for testing, or simulate login flows in staging. Also, if you use tiers for different levels of access for features, build a way to move users between tiers so that you can test feature access at all levels.

When Do You Outgrow Serverless Platforms Like Vercel?

Vercel is fantastic for static and dynamic frontends. But it's not a full backend, and serverless does come with tradeoffs especially if you're running anything more than light API calls.

What is Serverless Architecture?

Serverless architecture is a way of building apps where you don't manage servers yourself. Instead, you use services (like Vercel, AWS Lambda, or Supabase) that run your code or store your data on demand — and only when needed.

You still need servers behind the scenes, but you don't set them up, maintain them, or keep them running 24/7. The platform handles that automatically.

Key traits of serverless:

- Your code runs in small functions (like an API endpoint)
- These functions spin up automatically when triggered
- You only pay when they're used
- Great for things like: simple APIs, user authentication, form submissions, static sites

You build a contact form !' when someone submits it, a serverless function handles the request, sends an email, then shuts down. You never deal with the server at all. It's fast, scalable, and great for smaller apps — but has limitations for long-running tasks or complex backend logic.

Here are some limits to be aware of:

& p Cold Starts and Execution Limits

- Vercel's serverless functions have execution timeouts (typically 10 seconds)
- You may hit issues with cold starts (a delay before the server spins up)
- Long-running tasks like generating PDFs, uploading videos, API calls to a 3rd party service (like ChatGPT), or complex database queries may fail silently or time out

& p Limited Background Jobs

Need to send an email 2 minutes after signup? Or queue up a batch process? That's not really what Vercel was built for. You'll need:

- Something like Brevo to hold a contact list that can automatically send out emails when a new user is added to the list
 - Or a traditional backend with cron jobs (which is where platforms like Railway, Render, or Fly.io come in)
-

Why You Might Add a Platform Like Railway

Railway, Render, or even self-hosted Docker setups start to make sense when:

- You need longer running processes (eg. Make a call to ChatGPT's API to get answers to some data that you're analysing for a user)
- You want to run your own custom backend APIs (Express, FastAPI, etc.)
- You need full control over logs, environments, and background workers
- You're working with non-JavaScript tools, like Python, Go, or Rust backends

They're often used alongside Vercel where the frontend on Vercel, backend on Railway.

Example:

- Frontend: Vercel (React)
- Backend API: Railway (Node.js/Express)
- Database: Supabase
- Auth: Supabase
- Staging: duplicated Railway project and Supabase instance with separate keys

Database Considerations

Once you’re storing any real data, you’ll need a proper database and that comes with its own list of decisions.

Key things to think about:

- Separate staging and prod databases to avoid overwriting real data
- Do you want to host your own database or use managed services like Supabase or PlanetScale?
- How do you handle schema changes across environments?
- Do you need replication, backups, or regional availability?

Tools like Supabase are incredibly helpful but remember:

- Even their lowest plan (~\$10 SGD/month) only covers one project
- If you want staging + production, you’re doubling that cost
- You may also pay extra for bandwidth, storage, and auth limits so keep an eye on your usage.

Hidden (and Not-So-Hidden) Costs

Cost is a huge consideration when moving to a complex environment, especially for solo devs or small teams. Here's what to watch:

Service	What You Pay For	Starting Price (as of now)
Vercel	Usage, builds, bandwidth	Free !' \$20/mo (Pro)
Supabase	Project limits, row count, auth requests	\$10 SGD/project/mo
Railway	Resource usage (RAM, CPU, runtime hours)	Free tier, then \$20/mo
Email services (Brevo, Postmark)	Email sends	Often metered
Background jobs (Inngest, Trigger.dev)	Executions, workflows	Free !' usage-based

Multiply that if you run separate instances for staging and production.

(Pro tip: In early dev, keep staging + prod identical, but try to share free-tier limits if you can or toggle environments via env variables to avoid needing full duplicates right away.

Other Considerations for Complex Apps

Here’s a quickfire list of things that come into play as your app grows:

- Environment Variables: How do you manage different keys for dev, staging, and prod?
 - Secrets Management: Are your API keys safe, or hardcoded in plain text?
 - Monitoring: What happens if your app goes down? You might want tools like Sentry, LogRocket, or uptime pings.
 - Testing: Can you run automated tests in CI/CD before deploying?
 - Deployment Automation: Are you manually pushing to prod, or using GitHub workflows?
-

Final Thought

You definitely don't need all of this on day one, but knowing when and why to add it can save you from surprises down the road. If your app is growing beyond static content into something real and dynamic, especially with users, logins, and data. It's time to think about staging, hosting your own backend, and understanding the full cost picture.

The best part is: with AI support, you don't have to do it alone. You can prompt your way through every one of these decisions, and build an architecture that actually scales.